```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;



entity FIFO is

  Port (

      clk    : in STD_LOGIC;

      rst    : in STD_LOGIC;

      wren    : in STD_LOGIC;

      rden    : in STD_LOGIC;

      empty : out STD_LOGIC;
      full   : out STD_LOGIC;

      dataout : out STD_LOGIC_VECTOR(7 downto 0)

  );

end FIFO;
```

```
architecture Behavioral of FIFO is


 -- Clock divider signals

 signal clkout : STD_LOGIC := '0';

 signal counter : integer range 0 to 49999999 := 0;



 -- FIFO pointers and count

 signal rdptr : integer range 0 to 7 := 0;

 signal wrptr : integer range 0 to 7 := 0;
 signal count : integer range 0 to 8 := 0;


 -- FIFO memory: 8 locations, 8 bits each

 type fifo1 is array (0 to 7) of
     STD_LOGIC_VECTOR(7 downto 0);



 signal memory : fifo1 := (others => (others => '0'));
```

```
-- Input data

type datain1 is array (0 to 7) of

    STD_LOGIC_VECTOR(7 downto 0);




signal data : datain1 := (

    "10101010",

    "10111011",

    "10010101",
    "11110000",

    "00001111",

    "10100101",

    "11110101",

    "01100110"

);
```

```
begin



 -- Clock frequency division: 100 MHz to 1 Hz

 process(clk, rst)

 begin

     if rst = '1' then

       clkout <= '0';
       counter <= 0;



     elsif rising_edge(clk) then

       if counter = 49999999 then
      clkout <= not clkout;

      counter <= 0;

    else

      counter <= counter + 1;
    end if;
```

```
  end if;

end process;




-- Actual FIFO process using the divided clock

process(clkout, rst)

begin
  if rst = '1' then



    for i in 0 to 7 loop

       memory(i) <= (others => '0');

    end loop;



    rdptr <= 0;
    wrptr <= 0;
```

```
   count <= 0;

   empty <= '1';

   full   <= '0';

   dataout <= (others => '0');



elsif rising_edge(clkout) then


   -- Write operation

   if wren = '1' and rden = '0' then
if count < 8 then

   memory(wrptr) <= data(wrptr);



   if wrptr = 7 then
     wrptr <= 0;

   else

     wrptr <= wrptr + 1;
```

```
    end if;



    count <= count + 1;

    empty <= '0';


    if count = 7 then

      full <= '1';

    else

      full <= '0';

    end if;

  end if;


-- Read operation

elsif rden = '1' and wren = '0' then
```

```
if count > 0 then

  dataout <= memory(rdptr);



  if rdptr = 7 then
   rdptr <= 0;
  else

   rdptr <= rdptr + 1;

  end if;
        count <= count - 1;

        full <= '0';



        if count = 1 then

          empty <= '1';

        else

          empty <= '0';

        end if;
```

```
        end if;



      end if;



    end if;

  end process;



end Behavioral;
```